REVISION CHEAT SHEET:

SQL SERVER DATA TYPES

Interview-ready notes • examples • SQL • diagrams

A data type tells SQL Server what kind of value a column can store.

Example:

SQL

```
CREATE TABLE Employees
(
EmployeeId INT,
EmployeeName VARCHAR(100),
Salary DECIMAL(10,2),
BirthDate DATE
);
```

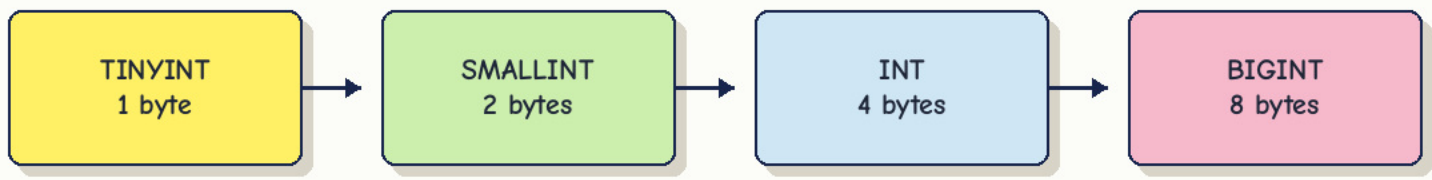
Here:

EXAMPLE / FLOW

```
INT          -> Whole number
VARCHAR      -> Text
DECIMAL      -> Exact decimal number
DATE         -> Date
```

Choosing the correct data type matters for storage, validation, performance, precision, and future growth.

Integer Numeric Types



Whole-number range grows -> storage grows

Integer types store whole numbers without decimal values.

SQL Server provides:

EXAMPLE / FLOW

```
TINYINT
SMALLINT
INT
BIGINT
```

Microsoft recommends using the smallest integer type that can safely hold all expected values.

TINYINT

Simple Definition

TINYINT stores very small positive whole numbers.

Range and Storage

EXAMPLE / FLOW

Range : 0 to 255
Storage : 1 byte

When to Use

- Small status numbers
- Rating values
- Small counters
- Month numbers
- Values guaranteed to stay between 0 and 255

When Not to Use

- Negative numbers
- Values greater than 255
- IDs that can grow significantly

Example

SQL

```
CREATE TABLE Students
(
StudentId INT,
Age TINYINT
);
```

SQL

```
INSERT INTO Students
VALUES (1, 25);
```

SMALLINT

Simple Definition

SMALLINT stores small positive or negative whole numbers.

Range and Storage

EXAMPLE / FLOW

Range:
-32,768 to 32,767

Storage:
2 bytes

When to Use

- Small counters
- Small numeric ranges

- Values that can exceed TINYINT
- Values guaranteed to remain below 32,768

When Not to Use

- IDs expected to grow significantly
- Values potentially larger than 32,767

Example

SQL

```
Temperature SMALLINT
```

Values such as:

EXAMPLE / FLOW

-100
500
10000

are valid.

INT

Simple Definition

INT is SQL Server's normal general-purpose whole-number type.

Range and Storage

EXAMPLE / FLOW

Range:
-2,147,483,648
to
2,147,483,647

Storage:
4 bytes

When to Use

- Employee IDs
- Product IDs
- Quantities
- Counters
- Most normal integer values

When Not to Use

If the number can exceed approximately:

EXAMPLE / FLOW

2.1 billion

use BIGINT.

Example

```sql
CREATE TABLE Products
(
ProductId INT,
StockQuantity INT
);
```

BIGINT

Simple Definition

BIGINT stores extremely large whole numbers.

Range and Storage

EXAMPLE / FLOW

Range:
-9,223,372,036,854,775,808
to
9,223,372,036,854,775,807

Storage:
8 bytes

When to Use

- Huge transaction tables
- Very large audit tables
- Large event systems
- IDs that may exceed the INT range

When Not to Use

Don't use it automatically for every number.

For example:

```sql
Age BIGINT
```

would be unnecessary.

Example

```sql
CREATE TABLE Transactions
(
TransactionId BIGINT
);
```

Integer Cheat Sheet

TypeRangeStorageCommon Use			
TINYINT	0 to 255	1 byte	Small status/rating
SMALLINT	-32,768 to 32,767	2 bytes	Small numbers
INT	±2.1 billion	4 bytes	Normal IDs/numbers
BIGINT	±9.22 quintillion	8 bytes	Huge IDs/counters

Memory:

EXAMPLE / FLOW

```
Very small   -> TINYINT
Small        -> SMALLINT
Normal       -> INT
Very large   -> BIGINT
```

Exact Numeric Types

Exact numeric values preserve the number accurately.

Important types:

EXAMPLE / FLOW

```
DECIMAL
NUMERIC
```

In SQL Server, DECIMAL and NUMERIC are functionally equivalent.

DECIMAL / NUMERIC

Simple Definition

Used for numbers where decimal precision must be exact.

Example:

EXAMPLE / FLOW

```
1250.75
99.99
18.50
```

Syntax

SQL

```
DECIMAL(precision, scale)
```

Example:

SQL

```
DECIMAL(10,2)
```

means:

EXAMPLE / FLOW

10 total digits

2 digits after decimal

Example value:

EXAMPLE / FLOW

12345678.99

SQL Server supports precision up to 38.

When to Use

- Price
- Salary
- Tax
- Financial calculations
- Exact measurements
- Percentages requiring exact values

When Not to Use

Don't use when approximate scientific floating-point behavior is specifically required.

Example

SQL

```
CREATE TABLE Products
(
ProductId INT,
Price DECIMAL(10,2)
);
```

SQL

```
INSERT INTO Products
VALUES (1, 1299.99);
```

Precision and Scale

Consider:

EXAMPLE / FLOW

12345.67

There are:

EXAMPLE / FLOW

7 total digits
2 digits after decimal

Therefore:

EXAMPLE / FLOW

Precision = 7
Scale = 2

So it fits:

SQL

```
DECIMAL(7,2)
```

Memory:

EXAMPLE / FLOW

DECIMAL(p,s)

p = Total digits
s = Digits after decimal

Approximate Numeric Types

Approximate numeric types store floating-point numbers.

They include:

EXAMPLE / FLOW

REAL
FLOAT

The stored result can be extremely close to the original number without being mathematically exact.

REAL

Simple Definition

REAL stores approximate floating-point numbers with lower precision than normal FLOAT.

Storage

EXAMPLE / FLOW

4 bytes

It provides roughly seven decimal digits of precision.

When to Use

- Scientific calculations
- Sensor readings
- Approximate measurements

When Not to Use

- Money
- Exact financial calculations
- Values requiring exact equality

Example

```sql
Temperature REAL
```

FLOAT

Simple Definition

FLOAT stores approximate floating-point numbers with greater range and precision.

Normal:

```sql
FLOAT
```

is equivalent to:

```sql
FLOAT(53)
```

and typically uses 8 bytes.

When to Use

- Scientific calculations
- Statistical calculations
- Approximate measurements
- Very large/small floating-point numbers

When Not to Use

Avoid:

```sql
Price FLOAT
```

Prefer:

```sql
Price DECIMAL(10,2)
```

when exact numbers matter.

Example

```sql
CREATE TABLE Measurements
(
Distance FLOAT
);
```

Microsoft recommends against FLOAT/REAL when exact numeric behavior, financial values, or reliable equality comparison is required.

Numeric Cheat Sheet

TypeExact?Storage / PrecisionCommon Use			
DECIMAL(p,s)	Yes	Depends on precision	Money/exact decimals
NUMERIC(p,s)	Yes	Same as DECIMAL	Same as DECIMAL
REAL	No	4 bytes / \~7 digits	Approximate values
FLOAT	No	Usually 8 bytes / \~15 digits	Scientific calculations

Memory:

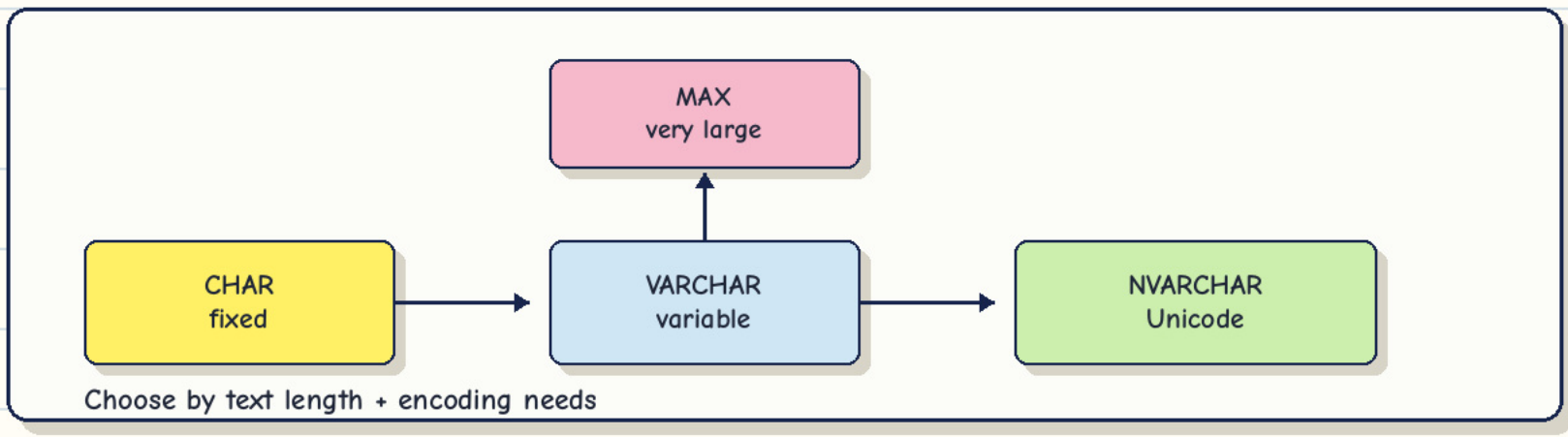
EXAMPLE / FLOW

DECIMAL
=
Exact

FLOAT
=
Approximate

Aa

Character / String Types



String types store text.

Important types:

EXAMPLE / FLOW

CHAR
VARCHAR
VARCHAR(MAX)

NCHAR
NVARCHAR
NVARCHAR(MAX)

TEXT

CHAR

Simple Definition

CHAR stores fixed-length text.

Syntax:

```sql
CHAR(n)
```

Regular CHAR supports up to 8,000 bytes.

When to Use

Values that are always approximately the same fixed length:

- Country code
- Fixed code
- Fixed indicator

When Not to Use

Don't use for highly variable text such as names.

Bad:

```sql
Name CHAR(100)
```

Better:

```sql
Name VARCHAR(100)
```

Example

```sql
CountryCode CHAR(2)
```

Values:

EXAMPLE / FLOW

US
IN
UK

Aa

VARCHAR

Simple Definition

VARCHAR stores variable-length character text.

Syntax:

```sql
VARCHAR(n)
```

Regular VARCHAR(n) supports up to 8,000 bytes.

When to Use

- Name
- Email
- Address
- Product name

- Normal variable text

When Not to Use

Don't use VARCHAR(MAX) automatically when the business length is known.

Example

```sql
FirstName VARCHAR(100)
```

Aa

VARCHAR(MAX)

Simple Definition

Stores very large variable-length text.

Maximum

Approximately:

EXAMPLE / FLOW

2 GB

When to Use

- Long descriptions
- Large text documents
- Large notes
- Very large textual payloads

When Not to Use

Bad:

```sql
FirstName VARCHAR(MAX)
```

Better:

```sql
FirstName VARCHAR(100)
```

Example

```sql
Description VARCHAR(MAX)
```

NCHAR

Simple Definition

NCHAR stores fixed-length Unicode text using SQL Server's Unicode character storage.

Syntax:

```sql
NCHAR(n)
```

n can normally be 1 through 4000 byte-pairs.

When to Use

- Fixed-length Unicode codes
- Fixed international strings

When Not to Use

Don't use for variable-length names or descriptions.

Example

```sql
LanguageCode NCHAR(2)
```

Aa

NVARCHAR

Simple Definition

NVARCHAR stores variable-length Unicode text.

When to Use

- International names
- Multiple languages
- Unicode application data
- Emoji or international characters

When Not to Use

Don't use oversized columns without a business reason.

Example

```sql
CustomerName NVARCHAR(150)
```

Unicode literal:

```sql
INSERT INTO Customers(CustomerName)
VALUES (N'こんにちは');
```

NVARCHAR(n) supports up to 4,000 byte-pairs.

Aa

NVARCHAR(MAX)

Simple Definition

Stores very large Unicode text.

When to Use

- Large multilingual documents
- Long comments
- Large international text

When Not to Use

Don't use MAX for ordinary names, email addresses, or codes.

Example

```sql
Comments NVARCHAR(MAX)
```

Aa

VARCHAR and Unicode

A useful modern SQL Server point:

Traditionally:

EXAMPLE / FLOW

VARCHAR
-> Non-Unicode/code-page data

NVARCHAR
-> Unicode

But SQL Server supports UTF-8 collations for VARCHAR, meaning VARCHAR can also store the full Unicode range under an appropriate UTF-8 collation.

For beginners, NVARCHAR remains an easy choice when multilingual data is required.

TEXT

Simple Definition

TEXT is an old data type for large text.

Modern Rule

Do not use TEXT for new development.

Use:

```sql
VARCHAR(MAX)
```

instead.

TEXT, NTEXT, and IMAGE are deprecated and planned for removal in a future SQL Server version.

Aa

Character Cheat Sheet

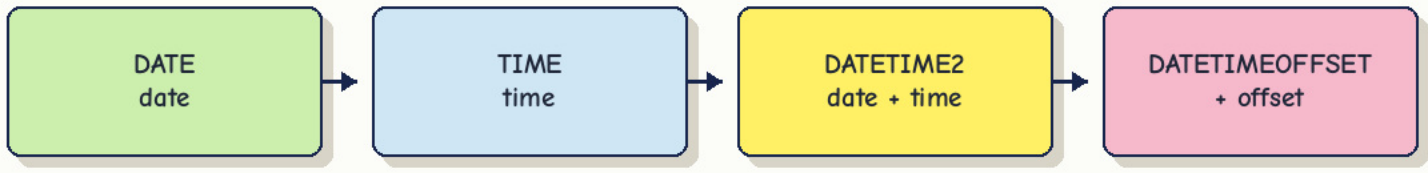
TypeSize BehaviorBest Use		
CHAR(n)	Fixed	Fixed codes
VARCHAR(n)	Variable	Normal text
VARCHAR(MAX)	Large variable	Large text
NCHAR(n)	Fixed Unicode	Fixed Unicode
NVARCHAR(n)	Variable Unicode	International text
NVARCHAR(MAX)	Large Unicode	Large international text
TEXT	Legacy	Avoid

Memory:

EXAMPLE / FLOW

CHAR -> Fixed
VARCHAR -> Variable
N -> Unicode-oriented
MAX -> Very large
TEXT -> Deprecated

Date and Time Types



Modern default for date + time: DATETIME2

Important types:

EXAMPLE / FLOW

DATE
TIME
DATETIME
DATETIME2
DATETIMEOFFSET

DATE

Simple Definition

Stores only a calendar date.

Example:

EXAMPLE / FLOW

2026-09-12

Range and Storage

EXAMPLE / FLOW

0001-01-01
to
9999-12-31

3 bytes

When to Use

- Birth date
- Hire date

- Due date
- Invoice date

When Not to Use

Don't use when time is also required.

Example

```sql
BirthDate DATE
```

TIME

Simple Definition

Stores only the time of day.

Example:

```
14:30:45
```

When to Use

- Opening time
- Closing time
- Daily schedule
- Appointment time when date is stored separately

When Not to Use

Don't use if you need the date too.

Example

```sql
StartTime TIME(0)
```

TIME supports up to seven fractional-second digits.

DATETIME

Simple Definition

Stores date and time together.

Example:

```
2026-09-12 14:30:45.123
```

Range

```
1753-01-01
through
9999-12-31
```

Storage

EXAMPLE / FLOW

8 bytes

When to Use

Mainly when working with older databases or existing columns already using DATETIME.

When Not to Use

Avoid for new development when DATETIME2 is suitable.

Microsoft explicitly recommends newer types such as DATE, TIME, DATETIME2, and DATETIMEOFFSET for new development.

DATETIME2

Simple Definition

Modern SQL Server type for storing date and time.

Example:

EXAMPLE / FLOW

2026-09-12 14:30:45.1234567

Range

EXAMPLE / FLOW

0001-01-01
through
9999-12-31

When to Use

- Created date/time
- Updated date/time
- Order timestamp
- Audit timestamp
- Most new date/time columns

When Not to Use

If you need to preserve a UTC offset, consider DATETIMEOFFSET.

Example

SQL

```
CreatedAt DATETIME2(3)
```

SQL

```
CreatedAt DATETIME2(3)
DEFAULT SYSUTCDATETIME()
```

DATETIMEOFFSET

Simple Definition

Stores:

EXAMPLE / FLOW

Date
+
Time
+
UTC offset

Example:

EXAMPLE / FLOW

2026-09-12 14:30:00 +05:30

When to Use

- Global applications
- Events originating in different offsets
- When original UTC offset matters

When Not to Use

If all timestamps are simply stored as UTC and you don't need the originating offset, DATETIME2 may be simpler.

Example

SQL

```
EventTime DATETIMEOFFSET(3)
```

SQL

```
SELECT SYSDATETIMEOFFSET();
```

Date/Time Cheat Sheet

TypeStoresGood For		
DATE	Date	Birth date
TIME	Time	Opening time
DATETIME	Date + time	Legacy systems
DATETIME2	Date + time	Modern applications
DATETIMEOFFSET	Date + time + offset	Global applications

Memory:

EXAMPLE / FLOW

DATE
-> Date

TIME
-> Time

DATETIME2
-> Date + time

DATETIMEOFFSET
-> Date + time + offset

For new development:

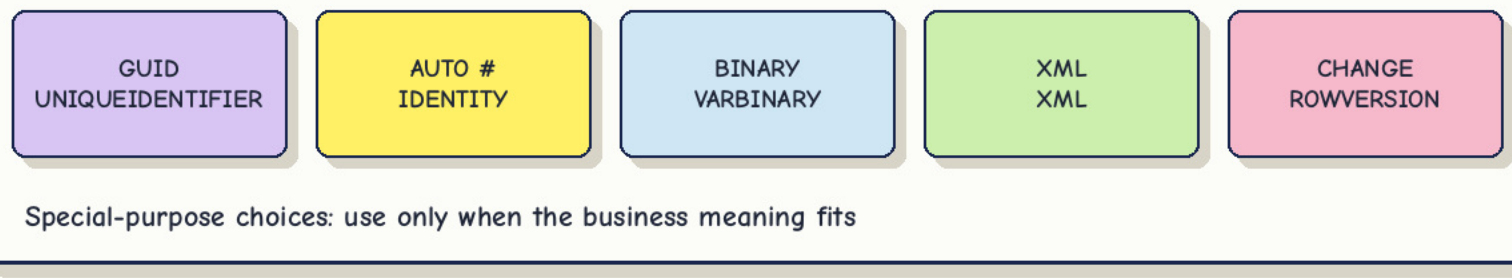
EXAMPLE / FLOW

Prefer DATETIME2 over DATETIME

in most normal situations.

0101

UNIQUEIDENTIFIER



Simple Definition

UNIQUEIDENTIFIER stores a GUID.

Example:

EXAMPLE / FLOW

6F9619FF-8B86-D011-B42D-00C04FC964FF

Storage

EXAMPLE / FLOW

16 bytes

When to Use

- Distributed systems
- Multiple systems generating IDs
- Globally unique public identifiers

When Not to Use

Don't automatically use it instead of INT or BIGINT.

GUIDs require more storage and create larger indexes.

Example

```sql
CREATE TABLE Customers
(
CustomerId UNIQUEIDENTIFIER
DEFAULT NEWID(),

CustomerName VARCHAR(100)
);
```

IDENTITY

Simple Definition

IDENTITY automatically generates numeric values.

Important:

> IDENTITY is a column property, not a data type.

Example:

```sql
CustomerId INT IDENTITY(1,1)
```

means:

```
INT
-> Data type

IDENTITY
-> Automatic number generation
```

Syntax

```sql
IDENTITY(seed, increment)
```

Example:

```sql
IDENTITY(1,1)
```

generates:

```
1
2
3
4
...
```

When to Use

- Auto-generated primary-key values
- Sequential internal IDs

When Not to Use

Don't use it when IDs must be generated independently across many disconnected systems.

Also don't assume identity values are always gap-free.

Example

SQL

```
CREATE TABLE Employees
(
EmployeeId INT IDENTITY(1,1) PRIMARY KEY,
EmployeeName VARCHAR(100)
);
```

0101

BINARY

Simple Definition

Stores fixed-length raw binary data.

Range

EXAMPLE / FLOW

```
BINARY(1)
through
BINARY(8000)
```

When to Use

- Fixed-size hashes
- Fixed binary values

When Not to Use

If binary values vary in size, use VARBINARY.

Example

SQL

```
HashValue BINARY(32)
```

0101

VARBINARY

Simple Definition

Stores variable-length binary data.

Size

EXAMPLE / FLOW

```
VARBINARY(1)
through
VARBINARY(8000)
```

When to Use

- Binary tokens
- Hashes
- Small binary payloads

When Not to Use

If values are larger than 8,000 bytes, use VARBINARY(MAX).

Example

SQL

```
EncryptedValue VARBINARY(500)
```

0101

VARBINARY(MAX)

Simple Definition

Stores very large binary values.

Maximum

Approximately:

EXAMPLE / FLOW

2 GB

When to Use

- PDFs
- Images
- Documents
- Large binary data

When Not to Use

For huge files, also consider whether external/object storage is architecturally more appropriate.

Example

SQL

```
FileData VARBINARY(MAX)
```

Aa

XML

Simple Definition

XML stores XML documents/data directly in SQL Server.

Example:

EXAMPLE / FLOW

```
<Customer>
<Name>John</Name>
</Customer>
```

When to Use

- XML messages
- XML configuration
- Existing XML document formats
- Semi-structured XML

When Not to Use

Don't put normal relational information into XML unnecessarily.

Bad design:

EXAMPLE / FLOW

CustomerId
Name
Email

all hidden inside XML when they need frequent searching and joining.

Example

SQL

```
CREATE TABLE Settings
(
SettingId INT,
Configuration XML
);
```

0101

ROWVERSION

Simple Definition

ROWVERSION is an automatically generated binary value that changes when the row is modified.

Important:

ROWVERSION does not store date or time.

A non-nullable ROWVERSION behaves like an automatically generated BINARY(8) value.

When to Use

- Optimistic concurrency
- Detecting whether another user changed a record

When Not to Use

Do not use it for:

EXAMPLE / FLOW

CreatedAt
UpdatedAt
Date/time tracking

Use DATETIME2 for those.

Example

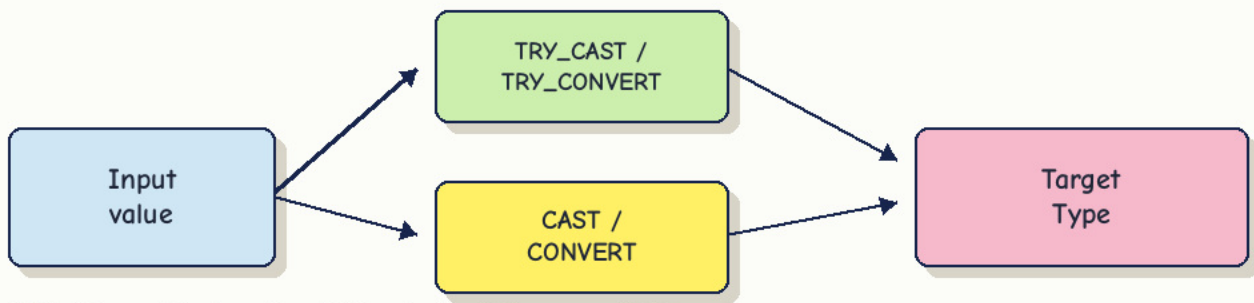
```sql
CREATE TABLE Products
(
ProductId INT PRIMARY KEY,
ProductName VARCHAR(100),
RowVersion ROWVERSION
);
```

0101

Special/Binary Cheat Sheet

TypeMain Purpose	
UNIQUEIDENTIFIER	GUID
IDENTITY	Auto-number property
BINARY	Fixed binary data
VARBINARY	Variable binary data
VARBINARY(MAX)	Large binary data
XML	XML data
ROWVERSION	Concurrency/change marker

Data Type Conversion



Sometimes one data type must be changed into another.

Example:

EXAMPLE / FLOW

```
VARCHAR
-> INT

INT
-> VARCHAR

VARCHAR
-> DATE

DECIMAL
-> INT
```

SQL Server supports:

EXAMPLE / FLOW

Implicit Conversion
Explicit Conversion

Implicit Conversion

SQL Server automatically converts the value when possible.

Example:

SQL

```
DECLARE @Number INT = 100;
SELECT @Number + 50;
```

SQL Server knows both values can participate in numeric arithmetic.

Another example:

SQL

```
DECLARE @Number INT = 100;
SELECT '100' + @Number;
```

Because INT has higher data-type precedence than VARCHAR, SQL Server can try converting '100' to INT. SQL Server uses documented data-type precedence rules when different types appear in the same expression.

Implicit conversions can sometimes hurt performance or cause errors, so don't depend on them unnecessarily.

Explicit Conversion

You explicitly tell SQL Server what conversion to perform.

Main functions:

EXAMPLE / FLOW

CAST()
CONVERT()
TRY_CAST()
TRY_CONVERT()

CAST

Syntax

SQL

```
CAST(expression AS data_type)
```

Example:

```sql
SELECT CAST('100' AS INT) AS Number;
```

Result:

```
100
```

Another:

```sql
SELECT CAST(125.99 AS INT);
```

Result:

```
125
```

The fractional portion is removed during this conversion.

CONVERT

Syntax

```sql
CONVERT(data_type, expression)
```

Example:

```sql
SELECT CONVERT(INT, '100');
```

Result:

```
100
```

CONVERT also supports a style argument, which is particularly useful for date formatting/conversion.

Example:

```sql
SELECT CONVERT(VARCHAR(10), GETDATE(), 23);
```

Possible result:

```
2026-09-12
```

CAST vs CONVERT

Both can perform conversions.

SQL

```
CAST('100' AS INT)
```

and:

SQL

```
CONVERT(INT, '100')
```

produce the same basic result.

Simple rule:

EXAMPLE / FLOW

CAST
-> Standard/simple conversion

CONVERT
-> SQL Server conversion with optional styles

TRY_CAST

TRY_CAST attempts a conversion.

If conversion fails, it normally returns:

EXAMPLE / FLOW

NULL

instead of raising a conversion error.

Example:

SQL

```
SELECT TRY_CAST('123' AS INT);
```

Result:

EXAMPLE / FLOW

123

But:

SQL

```
SELECT TRY_CAST('ABC' AS INT);
```

Result:

EXAMPLE / FLOW

NULL

TRY_CONVERT

Works similarly:

SQL

```
SELECT TRY_CONVERT(INT, '123');
```

Result:

EXAMPLE / FLOW

123

Invalid:

SQL

```
SELECT TRY_CONVERT(INT, 'ABC');
```

Result:

EXAMPLE / FLOW

NULL

When TRY_CAST / TRY_CONVERT Are Useful

Suppose imported data contains:

EXAMPLE / FLOW

100
200
ABC
500

Running:

SQL

```
CAST(Value AS INT)
```

could fail when it reaches:

EXAMPLE / FLOW

ABC

Instead:

```sql
TRY_CAST(Value AS INT)
```

returns:

```
100
200
NULL
500
```

This is useful for data validation and imports.

Common Conversion Examples

String to Integer

```sql
SELECT CAST('500' AS INT);
```

Result:

```
500
```

Integer to String

```sql
SELECT CAST(500 AS VARCHAR(10));
```

Result:

```
'500'
```

String to Decimal

```sql
SELECT CAST('125.50' AS DECIMAL(10,2));
```

Result:

```
125.50
```

String to Date

Prefer an unambiguous ISO-style date:

SQL

```
SELECT CAST('2026-09-12' AS DATE);
```

Result:

EXAMPLE / FLOW

2026-09-12

DATETIME2 to DATE

SQL

```
DECLARE @CreatedAt DATETIME2 =
'2026-09-12 14:30:45';

SELECT CAST(@CreatedAt AS DATE);
```

Result:

EXAMPLE / FLOW

2026-09-12

Conversion Cheat Sheet

RequirementExample	
String -> INT	CAST('100' AS INT)
INT -> String	CAST(100 AS VARCHAR(10))
String -> DECIMAL	CAST('10.50' AS DECIMAL(10,2))
String -> DATE	CAST('2026-09-12' AS DATE)
DATETIME2 -> DATE	CAST(CreatedAt AS DATE)
Safe conversion	TRY_CAST(value AS INT)
SQL Server-style conversion	CONVERT(INT, value)
Safe SQL Server conversion	TRY_CONVERT(INT, value)

Overall Data Type Cheat Sheet

RequirementType	
Tiny whole number	TINYINT
Small whole number	SMALLINT
Normal whole number	INT
Huge whole number	BIGINT
Exact decimal	DECIMAL
Approximate number	FLOAT
Fixed text	CHAR
Variable text	VARCHAR
Unicode text	NVARCHAR
Large text	VARCHAR(MAX) / NVARCHAR(MAX)
Date	DATE
Time	TIME
Date + time	DATETIME2
Date + time + offset	DATETIMEOFFSET
GUID	UNIQUEIDENTIFIER
Auto number	IDENTITY
Binary	VARBINARY
Large binary	VARBINARY(MAX)
XML	XML
Concurrency value	ROWVERSION

20 Interview Questions

1. What is a data type?

A data type defines what kind of value a SQL Server column can store and affects storage, validation, comparison, and operations.

2. What is the difference between TINYINT, SMALLINT, INT, and BIGINT?

They all store whole numbers but have different ranges and storage sizes. INT is the normal general-purpose choice, while BIGINT is used when values may exceed the INT range.

3. What is the range of TINYINT?

TINYINT stores values from 0 through 255 and uses one byte. It cannot store negative values.

4. When should BIGINT be used instead of INT?

Use BIGINT when values can exceed approximately 2.1 billion, such as IDs in extremely large transaction or event tables.

5. What is the difference between DECIMAL and NUMERIC?

There is no functional difference in SQL Server. They are synonyms and both provide fixed precision and scale.

6. What are precision and scale?

Precision is the total number of digits, while scale is the number after the decimal point. DECIMAL(10,2) allows 10 total digits and 2 after the decimal.

7. What is the difference between DECIMAL and FLOAT?

DECIMAL stores exact numeric values, while FLOAT stores approximate floating-point values. Use DECIMAL for financial values.

8. What is the difference between CHAR and VARCHAR?

CHAR is fixed-length, while VARCHAR is variable-length. Use CHAR for fixed codes and VARCHAR for variable text such as names.

9. What is the difference between VARCHAR and NVARCHAR?

NVARCHAR is SQL Server's UTF-16 Unicode type. VARCHAR uses the collation's encoding and can also support Unicode when using a UTF-8 collation.

10. Why shouldn't VARCHAR(MAX) be used everywhere?

MAX is intended for genuinely large data. Using appropriate bounded lengths better represents business rules and can simplify indexing and storage decisions.

11. Why should TEXT not be used?

TEXT is deprecated. New development should use VARCHAR(MAX); similarly, use NVARCHAR(MAX) instead of NTEXT.

12. What is the difference between DATE and DATETIME2?

DATE stores only the calendar date. DATETIME2 stores both date and time with configurable fractional-second precision.

13. DATETIME vs DATETIME2 - which should be used?

For new development, normally use DATETIME2. It has a larger date range, better precision, and configurable fractional seconds.

14. What is DATETIMEOFFSET?

DATETIMEOFFSET stores date, time, and UTC offset together. It is useful when an application's timestamps originate from multiple offsets.

15. What is UNIQUEIDENTIFIER?

UNIQUEIDENTIFIER is SQL Server's 16-byte GUID type. It is useful where globally unique identifiers are needed across distributed systems.

16. Is IDENTITY a data type?

No. IDENTITY is a column property used with numeric types such as INT or BIGINT to automatically generate values.

17. Does IDENTITY guarantee gap-free numbers?

No. Rollbacks, deletions, caching, and other operations can produce gaps. IDENTITY provides automatic generation, not gap-free numbering.

18. What is VARBINARY(MAX)?

VARBINARY(MAX) stores large variable-length binary values, such as documents or images, up to approximately 2 GB.

19. What is ROWVERSION?

ROWVERSION is an automatically generated 8-byte binary value that changes when a row is modified. It is commonly used for optimistic concurrency.

20. What is the difference between CAST and TRY_CAST?

CAST raises an error when an invalid conversion occurs. TRY_CAST normally returns NULL, making it useful when input data may contain invalid values.

Final Quick Revision

EXAMPLE / FLOW

Whole numbers
-> TINYINT / SMALLINT / INT / BIGINT

EXAMPLE / FLOW

Exact decimals
-> DECIMAL / NUMERIC

EXAMPLE / FLOW

Approximate numbers
-> REAL / FLOAT

EXAMPLE / FLOW

Text
-> CHAR / VARCHAR / NVARCHAR

EXAMPLE / FLOW

Large text
-> VARCHAR(MAX) / NVARCHAR(MAX)

EXAMPLE / FLOW

Date only
-> DATE

Time only
-> TIME

Modern date + time
-> DATETIME2

Date + time + offset
-> DATETIMEOFFSET

EXAMPLE / FLOW

GUID
-> UNIQUEIDENTIFIER

Auto number
-> IDENTITY

Binary
-> VARBINARY

XML
-> XML

Concurrency
-> ROWVERSION

EXAMPLE / FLOW

Conversion
-> CAST / CONVERT

Safe conversion
-> TRY_CAST / TRY_CONVERT

The main rule to remember is:

Choose the type that correctly represents the business value, safely supports its expected range and precision, and does not use unnecessary storage.